

The INT8 Configuration Cliff: Export Format and Graph Optimization, Not Weights, Govern a $40\times$ Latency Swing on Arm Cortex-A76

Manu Nicholas Jacob

Abstract—INT8 post-training quantization (PTQ) is widely assumed to accelerate CNN inference on edge CPUs, but the assumption hides a configuration dependence that dominates the result. On Arm Cortex-A76 (Raspberry Pi 5) with ONNX Runtime, we show that *identical* INT8 weights yield anywhere from a $10.7\times$ speedup to a $2.2\times$ slowdown relative to FP32, depending on two configuration choices that are invisible in point-estimate benchmarks: the ONNX quantization *export format* (QDQ vs. QOperator) and the runtime *graph-optimization level*. QDQ format at optimization level `ENABLE_ALL` gives $2.6\text{--}10.7\times$ speedups; the same QDQ model at `ENABLE_BASIC` is $2.1\times$ slower than FP32; and QOperator-format models are $1.8\text{--}4\times$ slower at every optimization level. Top-1 accuracy is preserved to within 0.5 points throughout, so the speedup—when the configuration is correct—is effectively free. We give the mechanism (QDQ requires extended-level fusion; QOperator’s QLinear kernels lose to the A76’s FP32 NEON path for small layers) and a two-line configuration rule, and argue that INT8 latency on edge CPUs is meaningless without its export format, optimization level, and runtime version pinned. All artifacts are released.

Index Terms—Edge inference, INT8 quantization, ONNX Runtime, Arm Cortex-A76, reproducibility.

I. INTRODUCTION

INT8 post-training quantization is a default optimization for deploying CNNs on single-board computers [1], [3]. The folklore is simple: quantize to INT8, get a smaller and faster model. Model size does shrink deterministically ($3.3\text{--}4.0\times$ here). Latency does not follow the folklore. On the same Raspberry Pi 5, with the same quantized weights, we measured INT8 inference ranging from $10.7\times$ faster to $2.2\times$ slower than FP32—a swing of roughly $40\times$ in the INT8/FP32 latency ratio—controlled entirely by two configuration choices that most benchmarks neither report nor vary.

The two choices are: (i) the ONNX *export format* of the quantized model—QDQ (QuantizeLinear/DequantizeLinear wrapper nodes) versus QOperator (fused QLinearConv); and (ii) the ONNX Runtime *graph-optimization level* (`DISABLE_ALL`/`BASIC`/`EXTENDED`/`ALL`). Neither is a property of the model weights, and neither appears in typical

latency tables. Yet together they decide whether INT8 is a large win or a net loss.

This letter (i) quantifies the configuration cliff across two architectures and two datasets; (ii) explains it mechanistically; and (iii) confirms that accuracy is unaffected, so the choice is purely a latency concern. Together these show why an INT8 latency number on an edge CPU is not reproducible unless its export format, optimization level, and runtime version are pinned.

II. EXPERIMENTAL SETUP

Hardware/software. Raspberry Pi 5, Arm Cortex-A76 quad-core @ 2.4 GHz, 64-bit Raspberry Pi OS, ONNX Runtime 1.24.4, CPUExecutionProvider, `intra_op_num_threads=4`. CPU governor performance; `vcgencmd get_throttled=0x0` verified (no throttling).

Models/datasets. MobileNetV3-Small (MV3, 1.6M params) and ResNet-18 (R18, 11.2M params), each trained on CIFAR-100 (100 classes) and SVHN (10 classes), input 32×32 RGB, exported to ONNX opset 17. INT8: static PTQ, MinMax calibration, per-channel weights.

Protocol. Latency: 40–60 warmup iterations discarded, 200–500 timed iterations via `time.perf_counter`; we report P50 and the mean. Accuracy: full held-out test sets (CIFAR-100 10k, SVHN 26k). We sweep the ONNX Runtime `graph_optimization_level` explicitly rather than relying on its default.

III. RESULTS

A. The configuration cliff

Table I reports FP32 and INT8 (QDQ format) P50 latency at optimization levels `BASIC` and `ALL` for all four model/dataset pairs. At `ALL`, INT8 is $2.6\times$ (MV3) to $10.7\times$ (R18) faster. At `BASIC`, the *same INT8 files* are $2.1\times$ slower than FP32. The only change is one runtime flag.

Table II and Fig. 1(a) sweep all four optimization levels for MV3/C100. INT8 is slower than FP32 at `DISABLE_ALL` and `BASIC`, and faster only from `EXTENDED` upward, where the QDQ nodes are fused. The transition is a cliff, not a gradient.

M.N.Jacob received dual B.S. degrees in Electrical Engineering and Computer Engineering from the University of Massachusetts Amherst, Amherst, MA, USA (2025). E-mail: manunicholasjacob@gmail.com. ORCID: 0009-0007-6589-6572. Code and data: <https://github.com/manunicholasjacob/rpi5-quantization-benchmark>.

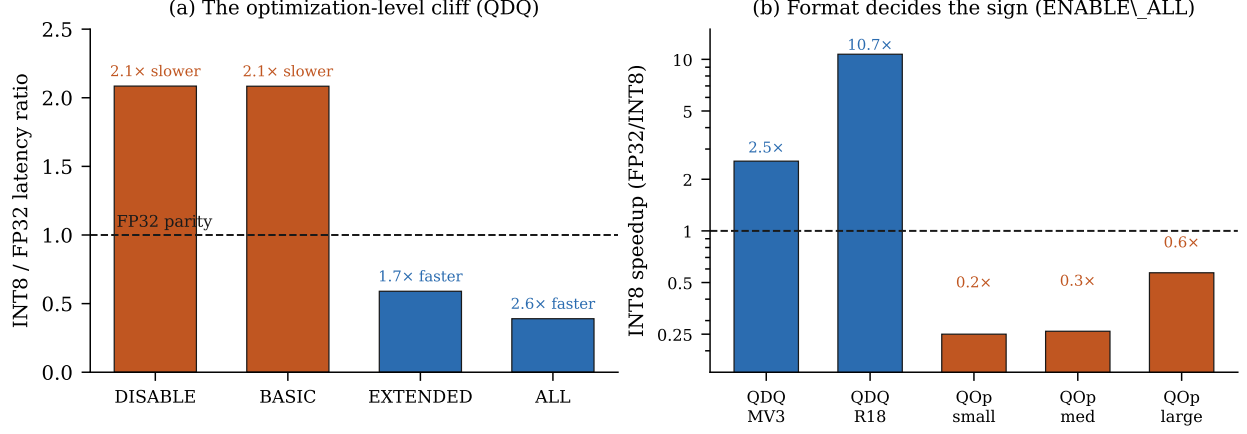


Fig. 1. The same INT8 weights span a $\sim 40\times$ latency range on Arm Cortex-A76, set by two configuration choices. (a) Optimization-level cliff: a QDQ-format INT8 model (MobileNetV3, CIFAR-100) is $2.1\times$ slower than FP32 below EXTENDED and $2.6\times$ faster at ENABLE_ALL, because QDQ node fusion only occurs at extended levels. (b) At ENABLE_ALL, export format decides the sign: QDQ models fuse and win ($2.5\text{--}10.7\times$), QOperator models do not and lose ($0.25\text{--}0.57\times$) regardless of level. Measured on Raspberry Pi 5, ONNX Runtime 1.24.4.

TABLE I

P50 LATENCY (MS), BATCH 1, 4 THREADS. INT8 MODELS ARE QDQ FORMAT. “SPD” IS FP32/INT8; >1 MEANS INT8 FASTER. SAME MODEL FILES IN BOTH INT8 COLUMNS.

Model	DS	FP32 (ALL)	INT8 @BASIC	spd @BASIC	INT8 @ALL	spd @ALL
MV3	C100	1.360	2.960	$0.46\times$	0.533	$2.55\times$
MV3	SVHN	1.353	2.857	$0.47\times$	0.521	$2.60\times$
R18	C100	11.746	18.306	$0.64\times$	1.097	$10.71\times$
R18	SVHN	11.639	18.502	$0.63\times$	1.089	$10.69\times$

TABLE II

INT8 (QDQ) vs. FP32, MV3/CIFAR-100, BY GRAPH-OPTIMIZATION LEVEL. RATIO = INT8/FP32 LATENCY; <1 MEANS INT8 FASTER.

Opt. level	FP32 (ms)	INT8 (ms)	INT8/FP32
DISABLE_ALL	1.407	2.933	$2.085\times$ (slower)
BASIC	1.391	2.898	$2.084\times$ (slower)
EXTENDED	1.372	0.809	$0.590\times$ (faster)
ALL	1.373	0.535	$0.390\times$ (faster)

B. Format matters even at ENABLE_ALL

The optimization level is not the whole story: the export *format* is decisive independently (Fig. 1(b)). Table III compares QDQ-format models (above) against QOperator-format INT8 models of a comparable MobileNetV3 early-exit backbone. QOperator models are $1.8\text{--}4.0\times$ slower than FP32 at every optimization level, including ALL: their QLinearConv kernels do not beat the A76’s FP32 NEON path for these small layers, and there is no fusion step that rescues them.

C. Accuracy is preserved

The latency swing carries no accuracy cost. Table IV reports top-1 accuracy on the full held-out test sets for FP32 and INT8 (QDQ). The drop is between -0.1 and $+0.4$ points across all pairs (and across 128/512/1024-sample calibration

TABLE III

EXPORT FORMAT DECIDES THE SIGN OF THE EFFECT AT ENABLE_ALL. QDQ FUSES AND WINS; QOPERATOR DOES NOT. “SPD” = FP32/INT8.

Format	Layer scale	INT8/FP32	spd@ALL
QDQ	MV3 full / R18 full	$0.39\text{--}0.09\times$	$2.6\text{--}10.7\times$
QOperator	MV3 exit1 (small)	$3.9\times$	$0.25\times$ (slower)
QOperator	MV3 exit2 (medium)	$3.9\times$	$0.26\times$ (slower)
QOperator	MV3 final (large)	$1.7\times$	$0.57\times$ (slower)

TABLE IV

TOP-1 ACCURACY (%), FULL TEST SETS. INT8 = QDQ. DROP IS WITHIN NOISE FOR EVERY PAIR.

Model	Dataset	FP32	INT8	Δ
MV3	CIFAR-100	29.06	28.63	-0.43
MV3	SVHN	86.21	85.86	-0.35
R18	CIFAR-100	46.37	46.44	$+0.07$
R18	SVHN	93.10	93.10	-0.00

variants, omitted for space). The configuration choice is therefore purely about latency: correct configuration yields a $2.6\text{--}10.7\times$ speedup at essentially no quality cost, while incorrect configuration forfeits it and adds a slowdown.

IV. DISCUSSION

Mechanism. ONNX Runtime executes QDQ models by fusing QuantizeLinear/DequantizeLinear pairs into integer kernels, but only at optimization level EXTENDED or ALL; below that, the graph runs quantize $\rightarrow$ dequantize $\rightarrow$ FP32-conv, which is strictly more work than plain FP32 (Table II). QOperator models ship pre-fused QLinearConv nodes, so they are optimization-level-insensitive, but their integer convolution kernels do not outperform the A76’s well-tuned FP32 NEON path for the small (≤ 2 M-element) layers typical of edge CNNs, leaving them net-slower (Table III).

Practical rule. On Arm Cortex-A76 with ONNX Runtime: (1) export INT8 in **QDQ** format; (2) set

`graph_optimization_level = ORT_ENABLE_ALL`. Both are required; either alone leaves INT8 slower than FP32. Report the export format and optimization level alongside every INT8 latency number, and pin the runtime version.

Scope. Cortex-A76 / ONNX Runtime / CPU provider / two CNN families / 32×32 inputs. TFLite/TVM backends, GPU/NPU providers, transformers, and larger inputs are out of scope; the qualitative lesson—that export format and optimization level can invert the sign of the INT8 effect—is expected to generalize, but the specific thresholds are platform-specific.

V. CONCLUSION

On the same Arm Cortex-A76 and the same INT8 weights, inference latency spans a $\sim 40\times$ range set entirely by the ONNX export format and the runtime graph-optimization level, with accuracy held constant to within 0.5 points. QDQ format at `ENABLE_ALL` delivers $2.6\text{--}10.7\times$ speedups; QOperator format, or any sub-EXTENDED optimization level, makes INT8 *slower* than FP32. The practical contribution is a pair of configuration rules and a warning: on edge CPUs, an INT8 latency number without its export format, optimization level, and runtime version pinned is not a reproducible measurement.

ACKNOWLEDGMENT

No external funding was received for this work.

REFERENCES

- [1] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper,” *arXiv:1806.08342*, 2018.
- [3] C. Banbury et al., “MLPerf Tiny Benchmark,” in *Proc. NeurIPS Datasets Benchmarks*, 2021.
- [4] ONNX Runtime developers, “ONNX Runtime,” 2024. [Online]. Available: <https://onnxruntime.ai>
- [5] A. Howard et al., “Searching for MobileNetV3,” in *Proc. IEEE/CVF ICCV*, 2019, pp. 1314–1324.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [7] R. David et al., “TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems,” in *Proc. MLSys*, 2021.
- [8] R. Banner, Y. Nahshan, and D. Soudry, “Post-Training 4-bit Quantization of Convolutional Networks for Rapid-Deployment,” in *Proc. NeurIPS*, 2019.
- [9] M. Nagel et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] D. Bacciu et al., “An Experimental Characterization of Deep Learning Inference on Edge Devices,” *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 2, pp. 1056–1071, 2021.